

SMAI – Complete Study Guide

Statistical Methods in Artificial Intelligence

IIIT Hyderabad • Jan–March 2026

Auto-generated from Lecture Notes

Last compiled: March 30, 2026

Contents

1	Introduction & Course Overview	3
1.1	Types of Machine Learning	3
1.2	Data Representation	3
1.3	Classification vs. Regression	4
1.4	Nearest Neighbour (NN) Classifier	4
2	Linear Regression	5
2.1	The Model	5
2.2	Loss Function: Mean Squared Error (MSE)	5
2.3	Closed-Form Solution (Normal Equation)	5
2.4	Gradient Descent	6
3	Regularization	7
3.1	Overfitting vs. Underfitting	7
3.2	L2 Regularization (Ridge Regression)	7
3.3	L1 Regularization (Lasso Regression)	7
3.4	L1 vs. L2 Comparison	8
4	Linear Algebra Foundations	9
4.1	Matrix Rank	9
4.2	Eigenvalues and Eigenvectors	9
4.3	Singular Value Decomposition (SVD)	10
4.4	Low-Rank Approximation	10
5	Principal Component Analysis (PCA)	11
5.1	Step-by-Step PCA Algorithm	11
5.2	The Covariance Matrix	11
5.3	Why Eigenvectors of Covariance Matrix?	11
5.4	Reconstruction from PCA	12
5.5	Variance Explained	12
5.6	PCA via SVD	12
5.7	Important Properties of PCA	12
5.8	Applications of PCA & SVD	12

6	Bayesian Decision Theory	14
6.1	Bayes' Theorem	14
6.2	Bayesian Decision Rule	14
6.3	Normal (Gaussian) Distribution	14
6.4	Bayesian Decision Boundary	15
7	Maximum Likelihood Estimation & Gaussian Mixture Models	16
7.1	Maximum Likelihood Estimation (MLE)	16
7.2	Gaussian Mixture Model (GMM)	16
7.3	Expectation-Maximization (EM) Algorithm	16
7.4	K-Means Clustering	17
8	Classification Algorithms	18
8.1	Naive Bayes Classifier	18
8.2	Logistic Regression	18
8.3	Softmax Regression (Multi-Class)	19
8.4	Loss Function Comparison	19
9	Support Vector Machines (SVM)	20
9.1	Geometric Intuition	20
9.2	Hard-Margin SVM	20
9.3	Soft-Margin SVM	20
9.4	The Kernel Trick	20
10	Practical Machine Learning & Model Evaluation	22
10.1	Data Splitting	22
10.2	Cross-Validation	22
10.3	Performance Metrics	22
10.4	Data Normalization	23
10.5	Hyperparameter Tuning	23
11	Neural Networks	24
11.1	Single-Layer Perceptron	24
11.2	Perceptron Learning Rule	24
11.3	Delta Rule (for Regression)	25
11.4	Activation Functions	25
11.5	Multi-Layer Perceptron (MLP)	25
11.6	Biological vs. Artificial Neurons	25
12	Neural Networks as Function Approximators (L15 – 9 March)	26
12.1	Why Non-Linearity is Essential	26
12.2	The XOR Problem Revisited	26
12.3	MLP Computation Flow	26
12.4	Universal Approximation Theorem	27
12.5	Softmax and Cross-Entropy Loss	27
12.6	Feature Hierarchy in Deep MLPs	27
13	Backpropagation Algorithm (L16 – 12 March)	29
13.1	Neural Network as a Computational Graph	29
13.2	Forward Pass	29
13.3	Backward Pass and the Chain Rule	29
13.4	Derivatives of Common Layers	30
13.5	Backpropagation Algorithm (General)	30

13.6 Vanishing and Exploding Gradients	30
13.7 Residual / Skip Connections	31
14 Neural Network Design & Training (L17 – 23 March)	32
14.1 Weight Initialisation	32
14.2 Gradient Descent Variants	32
14.3 Momentum	33
14.4 Regularisation for Neural Networks	33
14.5 Batch Normalisation	33
14.6 Learning Rate Scheduling	33
15 Transfer Learning & Advanced Optimisation (L18 – 26 March)	35
15.1 Transfer Learning	35
15.2 Advanced Gradient Descent: Motivation	35
15.3 Second-Order Methods	35
15.4 Adaptive Learning Rate Methods	36
15.5 Practical Training Checklist	36
16 Formula Reference Sheet	39
17 Practice Exam Questions	41
18 Quick-Reference: Topic-by-Lecture Map	44
19 Glossary of Key Terms	46

1 Introduction & Course Overview

What is this course about?

This course covers **Statistical Methods in AI (SMAI)**, split roughly 50:50 between:

1. **Classical / Basic Methods:** Linear regression, classification, regularization, PCA, SVD, Bayesian methods, SVMs.
2. **Neural Networks & Deep Learning:** Perceptrons, multi-layer networks, backpropagation, modern architectures.

1.1 Types of Machine Learning

Supervised Learning

The algorithm learns from **labelled data** — each training example comes with a correct answer (label).

Given $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$, learn $f : \mathbf{x} \mapsto y$.

Examples: spam detection, image classification, predicting house prices.

Unsupervised Learning

The algorithm learns from **unlabelled data** — no correct answers are provided. The goal is to discover hidden patterns, groupings, or structure.

Given $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$, find patterns.

Examples: clustering (K-means), dimensionality reduction (PCA), anomaly detection.

Self-Supervised Learning

A variant where the algorithm creates its own labels from the unlabelled data and uses them to learn useful representations. Very popular in modern NLP and computer vision.

1.2 Data Representation

Vectors as Data

Every data point (sample) is represented as a **feature vector**:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \in \mathbb{R}^d$$

where d is the **dimensionality** (number of features).

Example: Representing a student by features like height, weight, GPA, age gives $d = 4$.

Handcrafted vs. Learned Representations

- **Handcrafted:** Features designed by a human expert using domain knowledge (e.g., color histogram for images).

- **Learned:** Features automatically extracted from data by algorithms, especially neural networks (e.g., word embeddings, CNN features).

Modern deep learning mostly uses *learned* representations.

1.3 Classification vs. Regression

Aspect	Classification	Regression
Output type	Discrete / Categorical ($y \in \{0, 1, \dots, K\}$)	Continuous ($y \in \mathbb{R}$)
Goal	Predict a <i>class label</i>	Predict a <i>real number</i>
Example	Spam vs. Not Spam	Predict house price
Loss function	Cross-entropy, 0–1 loss	Mean Squared Error (MSE)

1.4 Nearest Neighbour (NN) Classifier

Nearest Neighbour Algorithm – The Simplest Classifier

Given a new test point $\mathbf{x}^*$:

1. Compute the distance between $\mathbf{x}^*$ and *every* training point.
2. Find the training point $\mathbf{x}_j$ that is closest (smallest distance).
3. Predict the label y_j of that nearest training point.

Distance metric: Typically Euclidean distance $\|\mathbf{x}^* - \mathbf{x}_j\|_2$.

Limitations of Nearest Neighbour

- **Slow at test time:** Must compare with *all* n training points — $O(nd)$ per prediction.
- **Curse of dimensionality:** Performance degrades in very high dimensions.
- **Stores entire dataset:** Memory-intensive.

2 Linear Regression

What is Linear Regression?

Linear regression is a **supervised learning** algorithm that predicts a **continuous** output by fitting a linear relationship between input features and the target variable.

2.1 The Model

Linear Regression Equation

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

- $\mathbf{w} = [w_1, w_2, \dots, w_d]^\top$ is the **weight vector** (parameters to learn).
- b is the **bias** (intercept).
- $\mathbf{x} = [x_1, x_2, \dots, x_d]^\top$ is the **input feature vector**.
- $\hat{y}$ is the **predicted output**.

Compact Notation

We often absorb the bias into the weight vector by appending a 1 to the feature vector:

$$\tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \quad \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}, \quad \hat{y} = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}.$$

2.2 Loss Function: Mean Squared Error (MSE)

Mean Squared Error

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \left(y_i - \mathbf{w}^\top \mathbf{x}_i \right)^2$$

This measures the average squared difference between predicted and actual values. Our goal is to **minimize** this loss.

2.3 Closed-Form Solution (Normal Equation)

Normal Equation

Stack all data into a matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ and vector $\mathbf{y} \in \mathbb{R}^n$. The optimal weights are:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

- This requires $\mathbf{X}^\top \mathbf{X}$ to be **invertible**.
- Computational cost: $O(d^3 + nd^2)$ — expensive when d is large.

2.4 Gradient Descent

Gradient Descent — Iterative Optimization

When the closed-form solution is expensive, we use **gradient descent**:

1. Initialize $\mathbf{w}$ randomly or as zeros.
2. Compute the gradient: $\nabla_{\mathbf{w}}\mathcal{L} = -\frac{2}{n}\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})$.
3. Update: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}}\mathcal{L}$, where η is the **learning rate**.
4. Repeat until convergence.

Variant	Description	Pros / Cons
Batch GD	Uses <i>all</i> n samples per update	Stable but slow for large n
Stochastic GD (SGD)	Uses <i>one</i> random sample per update	Fast but noisy
Mini-batch GD	Uses a small batch of B samples	Best trade-off in practice

Closed-Form vs. Gradient Descent

	Closed-Form	Gradient Descent
Speed	One-shot computation	Iterative
Memory	Needs to store $\mathbf{X}^\top\mathbf{X}$	Can work on mini-batches
Scalability	Poor when d or n large	Scales well
Parallelizable	Yes (matrix operations)	Yes (each sample independent)

3 Regularization

Why Regularization?

Regularization is a technique to **prevent overfitting** by adding a **penalty term** to the loss function. This discourages the model from learning overly complex patterns that fit noise in the training data.

$$\mathcal{L}_{\text{regularized}} = \underbrace{\mathcal{L}_{\text{data}}}_{\text{fit the data}} + \underbrace{\lambda \cdot R(\mathbf{w})}_{\text{keep weights small}}$$

where $\lambda > 0$ is the **regularization strength** (hyperparameter).

3.1 Overfitting vs. Underfitting

The Central Problem of Machine Learning

- **Overfitting (High Variance)**: Model is too complex. Fits training data perfectly but fails on new data. Training error $\ll$ test error.
- **Underfitting (High Bias)**: Model is too simple. Cannot capture the underlying patterns. Both training error and test error are high.

Goal: Find the sweet spot between bias and variance — the **bias–variance trade-off**.

3.2 L2 Regularization (Ridge Regression)

Ridge Regression

Add the squared magnitude of weights as penalty:

$$\mathcal{L}_{\text{Ridge}} = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_2^2$$

where $\|\mathbf{w}\|_2^2 = w_1^2 + w_2^2 + \dots + w_d^2$.

Closed-form solution: $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$

Effect: Shrinks all weights towards zero, but does *not* set them exactly to zero.

Geometric interpretation: Constraint region is a **circle** (or hypersphere).

3.3 L1 Regularization (Lasso Regression)

Lasso Regression

Add the absolute value of weights as penalty:

$$\mathcal{L}_{\text{Lasso}} = \frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \lambda \|\mathbf{w}\|_1$$

where $\|\mathbf{w}\|_1 = |w_1| + |w_2| + \dots + |w_d|$.

Effect: Can set some weights *exactly to zero* $\Rightarrow$ **feature selection** (sparse model).

Geometric interpretation: Constraint region is a **diamond** (in 2D).

3.4 L1 vs. L2 Comparison

Property	L1 (Lasso)	L2 (Ridge)
Penalty term	$\lambda \sum w_j $	$\lambda \sum w_j^2$
Sparsity	Yes — some weights become 0	No — weights are small but non-zero
Feature selection	Built-in	Not directly
Constraint shape	Diamond	Circle
Differentiable?	No (at $w_j = 0$)	Yes, everywhere
Best for	Many irrelevant features	All features contribute somewhat

4 Linear Algebra Foundations

4.1 Matrix Rank

Rank of a Matrix

The **rank** of a matrix $\mathbf{A}$ is the maximum number of **linearly independent** rows (or columns).

- If $\mathbf{A} \in \mathbb{R}^{m \times n}$, then $\text{rank}(\mathbf{A}) \leq \min(m, n)$.
- Rank = number of **non-zero singular values**.
- Rank = number of **non-zero eigenvalues** (for square matrices).

Linear Independence

Vectors $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ are **linearly independent** if no one vector can be written as a linear combination of the others. Formally:

$$c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \dots + c_k \mathbf{v}_k = \mathbf{0} \implies c_1 = c_2 = \dots = c_k = 0.$$

4.2 Eigenvalues and Eigenvectors

Eigenvalue Equation

For a square matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$, a scalar λ and non-zero vector $\mathbf{v}$ satisfying

$$\boxed{\mathbf{A}\mathbf{v} = \lambda\mathbf{v}}$$

are called an **eigenvalue** (λ) and **eigenvector** ($\mathbf{v}$) of $\mathbf{A}$.

What do eigenvalues/vectors mean?

- An **eigenvector** gives a *direction* that is unchanged by the transformation $\mathbf{A}$ (only scaled).
- The corresponding **eigenvalue** tells you *how much* the vector is stretched or shrunk along that direction.
- If $\lambda > 1$: stretching. If $0 < \lambda < 1$: shrinking. If $\lambda < 0$: flipping.

Eigendecomposition

If $\mathbf{A}$ is a symmetric matrix, it can be decomposed as:

$$\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^\top$$

where:

- $\mathbf{Q}$ is an orthogonal matrix whose columns are eigenvectors.
- $\mathbf{\Lambda}$ is a diagonal matrix of eigenvalues: $\mathbf{\Lambda} = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_d)$.

4.3 Singular Value Decomposition (SVD)

SVD – The Most Important Decomposition

Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be decomposed as:

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$$

where:

- $\mathbf{U} \in \mathbb{R}^{m \times m}$: orthogonal matrix (left singular vectors).
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$: diagonal matrix of **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.
- $\mathbf{V} \in \mathbb{R}^{n \times n}$: orthogonal matrix (right singular vectors).

Key Properties of SVD

- SVD works for **any** matrix (not just square or symmetric).
- The number of non-zero singular values = rank of $\mathbf{A}$.
- Singular values are always ≥ 0 .
- Relationship to eigenvalues: $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^\top \mathbf{A})}$.
- Can be used to compute pseudo-inverses, low-rank approximations, and more.

4.4 Low-Rank Approximation

Low-Rank Approximation via SVD

Given $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$, the best rank- k approximation (in Frobenius norm) is:

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

This is the **Eckart–Young theorem**. We keep only the top k singular values and their vectors, discarding the rest as “noise.”

Applications: Image compression, denoising, recommendation systems, latent semantic analysis.

Choosing k — How many components to keep?

Plot the singular values $\sigma_1 \geq \sigma_2 \geq \dots$. Look for a **sharp drop** (“elbow”) in the spectrum. Components after the elbow mostly capture noise.

5 Principal Component Analysis (PCA)

What is PCA?

PCA is the most popular **unsupervised dimensionality reduction** technique. It finds new axes (called **principal components**) along which the data has **maximum variance**.

Goal: Reduce dimensionality from d to d' (where $d' \ll d$) while retaining as much information as possible.

5.1 Step-by-Step PCA Algorithm

PCA Algorithm

1. **Center the data:** Subtract the mean: $\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$, then $\tilde{\mathbf{x}}_i = \mathbf{x}_i - \bar{\mathbf{x}}$.

2. **Compute the covariance matrix:**

$$\Sigma = \frac{1}{n} \sum_{i=1}^n \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_i^\top = \frac{1}{n} \tilde{\mathbf{X}}^\top \tilde{\mathbf{X}} \in \mathbb{R}^{d \times d}$$

3. **Compute eigenvalues and eigenvectors of Σ :** $\Sigma \mathbf{v}_j = \lambda_j \mathbf{v}_j$.

4. **Sort** eigenvectors by decreasing eigenvalue: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$.

5. **Select top d' eigenvectors:** Form $\mathbf{U} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_{d'}] \in \mathbb{R}^{d \times d'}$.

6. **Project:** $\mathbf{x}'_i = \mathbf{U}^\top \tilde{\mathbf{x}}_i \in \mathbb{R}^{d'}$.

5.2 The Covariance Matrix

Covariance Matrix – Intuition

The covariance matrix Σ captures how features **vary together**:

$$\Sigma_{jk} = \frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

- **Diagonal entries** Σ_{jj} = variance of feature j .
- **Off-diagonal entries** Σ_{jk} = covariance between features j and k .
- Σ is always **symmetric** and **positive semi-definite**.

5.3 Why Eigenvectors of Covariance Matrix?

PCA = Max Variance Directions

The first principal component $\mathbf{v}_1$ is the direction that maximizes the variance of the projected data:

$$\mathbf{v}_1 = \arg \max_{\|\mathbf{v}\|=1} \text{Var}(\mathbf{v}^\top \tilde{\mathbf{X}}) = \arg \max_{\|\mathbf{v}\|=1} \mathbf{v}^\top \Sigma \mathbf{v}$$

The solution is the **eigenvector** of Σ corresponding to the **largest eigenvalue**.

Each subsequent principal component is the direction of maximum variance *orthogonal to all previous ones*.

5.4 Reconstruction from PCA

Reconstruction

After projecting $\mathbf{x}'_i = \mathbf{U}^\top \tilde{\mathbf{x}}_i$, we can reconstruct approximately:

$$\hat{\mathbf{x}}_i = \mathbf{U} \mathbf{x}'_i + \bar{\mathbf{x}}$$

This reconstruction is **lossy** if $d' < d$ because we discarded some eigenvectors (those with small eigenvalues).

5.5 Variance Explained

Proportion of Variance Explained

The fraction of total variance captured by the top d' components is:

$$\text{Variance Explained} = \frac{\sum_{j=1}^{d'} \lambda_j}{\sum_{j=1}^d \lambda_j}$$

Common rule of thumb: Keep enough components to explain **90–95%** of the variance.

5.6 PCA via SVD

Connection: PCA and SVD

Given centered data matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{n \times d}$ with SVD: $\tilde{\mathbf{X}} = \mathbf{U}_X \Sigma_X \mathbf{V}_X^\top$:

- The right singular vectors $\mathbf{V}_X$ are the **principal components** (eigenvectors of $\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}}$).
- The singular values relate to eigenvalues: $\lambda_j = \frac{\sigma_j^2}{n}$.
- SVD is often more **numerically stable** and efficient than explicitly computing the covariance matrix.

5.7 Important Properties of PCA

PCA Gotchas

- PCA is **sensitive to scale**. Always **standardize** (zero mean, unit variance) features before PCA.
- PCA is a **linear** technique — it can only capture linear relationships.
- PCA assumes the directions of maximum variance are the most “interesting.”
- For non-linear dimensionality reduction, consider **t-SNE** or **UMAP**.

5.8 Applications of PCA & SVD

Real-World Applications

- **Image Compression**: Represent images using top- k principal components $\Rightarrow$ fewer bytes.
- **Face Recognition (Eigenfaces)**: PCA on face images finds the main “face directions.”

- **Denoising:** Low-rank approximation via SVD removes noise from data.
- **Latent Semantic Indexing (LSI):** Apply SVD to term-document matrix to find latent topics.
- **Word Embeddings:** Dense vector representations of words capturing semantic meaning.
- **Recommendation Systems:** Matrix factorization (SVD-based) for collaborative filtering (e.g., Netflix).
- **Data Visualization:** Project high-dimensional data to 2D / 3D for plotting.
- **Preprocessing for ML:** Reduce features before feeding into classifiers to reduce overfitting.

6 Bayesian Decision Theory

The Bayesian Approach

Bayesian decision theory provides a **principled framework** for making optimal decisions under **uncertainty**. The key idea: use **probability** to represent uncertainty and **Bayes' theorem** to update beliefs when new evidence arrives.

6.1 Bayes' Theorem

Bayes' Theorem – The Foundation

$$P(\omega_j | \mathbf{x}) = \frac{P(\mathbf{x} | \omega_j) P(\omega_j)}{P(\mathbf{x})}$$

where:

- $P(\omega_j | \mathbf{x})$ = **Posterior**: probability of class ω_j given observation $\mathbf{x}$.
- $P(\mathbf{x} | \omega_j)$ = **Likelihood**: probability of observing $\mathbf{x}$ if it belongs to class ω_j .
- $P(\omega_j)$ = **Prior**: probability of class ω_j before seeing $\mathbf{x}$.
- $P(\mathbf{x})$ = **Evidence**: total probability of observing $\mathbf{x}$ (normalization constant).

When to Remember

$$\text{Posterior} = \frac{\text{Likelihood} \times \text{Prior}}{\text{Evidence}}$$

6.2 Bayesian Decision Rule

Optimal Decision

Classify $\mathbf{x}$ as class ω_j that maximizes the posterior:

$$\hat{y} = \arg \max_j P(\omega_j | \mathbf{x})$$

Since $P(\mathbf{x})$ is the same for all classes, this is equivalent to:

$$\hat{y} = \arg \max_j P(\mathbf{x} | \omega_j) P(\omega_j)$$

This decision rule **minimizes the probability of error**.

6.3 Normal (Gaussian) Distribution

Univariate Normal Distribution

A random variable X follows a normal distribution $\mathcal{N}(\mu, \sigma^2)$ if its PDF is:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

- μ = **mean** (center of the bell curve).
- σ^2 = **variance** (spread of the bell curve).

- σ = standard deviation.

Multivariate Normal Distribution

For a d -dimensional random vector $\mathbf{x}$:

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

- $\boldsymbol{\mu} \in \mathbb{R}^d$ = **mean vector**.
- $\Sigma \in \mathbb{R}^{d \times d}$ = **covariance matrix** (symmetric, positive definite).
- $|\Sigma|$ = determinant of Σ .

6.4 Bayesian Decision Boundary

Decision Boundary Between Two Classes

For two Gaussian classes ω_1, ω_2 with parameters $(\boldsymbol{\mu}_1, \Sigma_1)$ and $(\boldsymbol{\mu}_2, \Sigma_2)$:

Case 1: Same covariance ($\Sigma_1 = \Sigma_2 = \Sigma$):

The decision boundary is **linear** (a hyperplane).

Case 2: Different covariances ($\Sigma_1 \neq \Sigma_2$):

The decision boundary is **quadratic** (a curve / ellipse).

7 Maximum Likelihood Estimation & Gaussian Mixture Models

7.1 Maximum Likelihood Estimation (MLE)

What is MLE?

Given observed data $\{x_1, x_2, \dots, x_n\}$ and a parametric model (e.g., Gaussian with unknown μ, σ^2), MLE finds the parameter values that make the observed data **most probable**.

MLE for Normal Distribution

Likelihood function (assuming i.i.d. samples):

$$L(\mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

Log-likelihood (easier to optimize):

$$\ell(\mu, \sigma^2) = -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

MLE solutions (set derivatives to zero):

$$\hat{\mu}_{\text{MLE}} = \frac{1}{n} \sum_{i=1}^n x_i \quad \hat{\sigma}_{\text{MLE}}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2$$

7.2 Gaussian Mixture Model (GMM)

GMM – Modeling Complex Distributions

A GMM models the data as coming from a **mixture of K Gaussian distributions**:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- π_k = **mixing coefficient** (weight) of component k . Must satisfy $\sum_k \pi_k = 1$, $\pi_k \geq 0$.
- $\boldsymbol{\mu}_k$ = mean of component k .
- $\boldsymbol{\Sigma}_k$ = covariance of component k .

Applications: clustering, density estimation, anomaly detection.

7.3 Expectation-Maximization (EM) Algorithm

EM Algorithm for GMM

Goal: Find parameters $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$ that maximize the likelihood of the data.

Iterate until convergence:

E-Step (Expectation): Compute the “responsibility” of component k for data point i :

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

M-Step (Maximization): Update parameters:

$$N_k = \sum_{i=1}^n \gamma_{ik}, \quad \boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{i=1}^n \gamma_{ik} \mathbf{x}_i$$
$$\boldsymbol{\Sigma}_k = \frac{1}{N_k} \sum_{i=1}^n \gamma_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^\top, \quad \pi_k = \frac{N_k}{n}$$

EM Caveats

- EM converges to a **local maximum**, not necessarily the global one.
- **Initialization matters**: Different starting points can give different solutions. Run multiple times.
- Must choose K (number of components) — use BIC, AIC, or cross-validation.

7.4 K-Means Clustering

K-Means — A Simpler Clustering Algorithm

K-means is a **hard clustering** algorithm (each point belongs to exactly one cluster). It can be seen as a special case of GMM with equal, spherical covariances.

Algorithm:

1. **Initialize** K cluster centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$ (randomly).
2. **Assign** each point to the nearest centroid: $c_i = \arg \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$.
3. **Update** centroids: $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$.
4. **Repeat** steps 2–3 until assignments stop changing.

8 Classification Algorithms

8.1 Naive Bayes Classifier

Naive Bayes

A probabilistic classifier based on Bayes' theorem with the **naive** assumption that all features are **conditionally independent** given the class:

$$P(\mathbf{x} \mid \omega_j) = \prod_{i=1}^d P(x_i \mid \omega_j)$$

Decision rule:

$$\hat{y} = \arg \max_j P(\omega_j) \prod_{i=1}^d P(x_i \mid \omega_j)$$

Pros: Simple, fast, works well with high-dimensional data (e.g., text).

Cons: Independence assumption is often violated in practice.

Applications: Spam filtering, text classification, sentiment analysis.

8.2 Logistic Regression

Logistic Regression – Despite the Name, It's for Classification!

Logistic regression models the **probability** that $\mathbf{x}$ belongs to class 1:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the **sigmoid (logistic) function**.

Sigmoid Function Properties

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Output is always in $(0, 1)$ — can be interpreted as a probability.
- $\sigma(0) = 0.5$.
- $\sigma(-z) = 1 - \sigma(z)$.
- Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Cross-Entropy Loss for Logistic Regression

$$\mathcal{L}(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n [y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i)]$$

where $\hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$. Minimize via gradient descent.

8.3 Softmax Regression (Multi-Class)

Softmax Function

For K classes, the probability of class k is:

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^K e^{\mathbf{w}_j^\top \mathbf{x}}}$$

This generalizes the sigmoid to multiple classes. The loss is **categorical cross-entropy**.

8.4 Loss Function Comparison

Loss Function	Description	Used For
0-1 Loss	$\ell = \mathbb{I}[\hat{y} \neq y]$	Ideal but non-differentiable
Hinge Loss	$\ell = \max(0, 1 - y \cdot f(x))$	SVM
Logistic Loss	$\ell = \ln(1 + e^{-y \cdot f(x)})$	Logistic Regression
Cross-Entropy	$-\sum y_k \ln \hat{p}_k$	Multi-class classification
MSE	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Regression

9 Support Vector Machines (SVM)

What is an SVM?

A Support Vector Machine finds the **hyperplane** that **maximally separates** two classes. Instead of just finding *any* separating hyperplane, SVM finds the one with the **largest margin**.

9.1 Geometric Intuition

Margin

The **margin** is the distance between the decision boundary (hyperplane) and the closest data points from either class. These closest points are called **support vectors**.

SVM objective: Maximize the margin. A larger margin means better generalization.

9.2 Hard-Margin SVM

Hard-Margin SVM Formulation

Assume data is linearly separable. Find $\mathbf{w}$ and b that:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i$$

- The margin width is $\frac{2}{\|\mathbf{w}\|}$.
- Minimizing $\|\mathbf{w}\|^2 \Leftrightarrow$ maximizing the margin.
- Constraints ensure all points are correctly classified with margin ≥ 1 .

9.3 Soft-Margin SVM

Soft-Margin SVM – Handling Non-Separable Data

Allow some misclassifications by introducing **slack variables** $\xi_i \geq 0$:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- $C > 0$ = regularization parameter controlling the trade-off between margin and errors.
- Large C : fewer errors allowed (could overfit).
- Small C : wider margin but more errors allowed (could underfit).

9.4 The Kernel Trick

Kernel Trick – Making SVM Non-Linear

When data is **not linearly separable**, we map it to a **higher-dimensional space** where it *is* separable. The kernel trick lets us do this **without explicitly computing the mapping**.

A **kernel function** $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$ computes the inner product in the high-

dimensional space directly.

Kernel	Formula
Linear	$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$
Polynomial (degree p)	$K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j + c)^p$
RBF / Gaussian	$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\sigma^2}\right)$

The XOR Problem

XOR is **not linearly separable** in 2D. But mapping to a higher dimension using a polynomial kernel makes it separable. This is a classic example of why kernels are powerful.

SVM Advantages & Disadvantages

Advantages:

- Works well in high-dimensional spaces.
- Effective when number of features $>$ number of samples.
- Memory efficient (only stores support vectors).
- Convex optimization — guaranteed global optimum (for given kernel).

Disadvantages:

- Doesn't directly output probabilities.
- Slow for very large datasets.
- Sensitive to feature scaling.
- Kernel choice requires experimentation.

10 Practical Machine Learning & Model Evaluation

The Most Important Lesson

Theory alone is not enough! You need hands-on practice. There is no single “best” algorithm — it depends on the data, the problem, and the computational resources.

10.1 Data Splitting

Train / Validation / Test Split

1. **Training Set** (e.g., 60–70%): Used to **train** the model (fit parameters).
2. **Validation Set** (e.g., 15–20%): Used to **tune hyperparameters** and **select the best model**.
3. **Test Set** (e.g., 15–20%): Used **only once at the very end** to report final performance.

Golden Rule: Never let the model see the test set during training or tuning!

10.2 Cross-Validation

k -Fold Cross-Validation

1. Split training data into k equal folds.
2. For each fold i : train on the other $k - 1$ folds, validate on fold i .
3. Average the k validation scores.

Typical: $k = 5$ or $k = 10$.

Benefit: Uses all data for both training and validation; more reliable estimate of performance.

10.3 Performance Metrics

Metric	Description
Accuracy	$\frac{\text{Correct predictions}}{\text{Total predictions}}$ — can be misleading with imbalanced classes
Precision	$\frac{TP}{TP+FP}$ — of those predicted positive, how many are actually positive?
Recall (Sensitivity)	$\frac{TP}{TP+FN}$ — of the actual positives, how many did we catch?
F1 Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ — harmonic mean of precision and recall

Confusion Matrix

A table showing actual vs. predicted classes:

	Predicted +	Predicted −
Actual +	True Positive (TP)	False Negative (FN)
Actual −	False Positive (FP)	True Negative (TN)

10.4 Data Normalization

Why Normalize?

Features can have very different scales (e.g., age: 0–100, salary: 0–1,000,000). Algorithms like gradient descent, SVM, and PCA are sensitive to scale.

Common methods:

- **Min-Max Scaling:** $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1]$.
- **Standardization (Z-score):** $x' = \frac{x - \mu}{\sigma}$ (zero mean, unit variance).

10.5 Hyperparameter Tuning

Common Hyperparameters

Algorithm	Key Hyperparameters
Linear Regression	Regularization λ
Logistic Regression	Regularization λ , learning rate η
SVM	C , kernel type, σ (for RBF)
K-Means	K (number of clusters)
PCA	d' (number of components)
Neural Network	Learning rate, number of layers, number of neurons, batch size

11 Neural Networks

What is a Neural Network?

A neural network is a **computational model** loosely inspired by biological neural networks. It is a **function** that maps inputs to outputs through a series of **layers** of interconnected “neurons.”

Key insight: Neural networks are **universal function approximators** — they can learn almost any mapping from inputs to outputs given enough data and capacity.

11.1 Single-Layer Perceptron

The Perceptron – The Simplest Neural Network

A single neuron computes:

$$\hat{y} = \text{activation}\left(\sum_{j=1}^d w_j x_j + b\right) = \text{activation}(\mathbf{w}^\top \mathbf{x} + b)$$

- **Inputs:** $x_1, x_2, \dots, x_d$.
- **Weights:** $w_1, w_2, \dots, w_d$ (learned from data).
- **Bias:** b .
- **Activation function:** Introduces non-linearity (e.g., step function, sigmoid, ReLU).
- **Output:** $\hat{y}$.

Limitation of Single-Layer Perceptron

A single-layer perceptron can only learn **linearly separable** functions. It **cannot** learn XOR!

Solution: Use **multi-layer** networks with non-linear activations.

11.2 Perceptron Learning Rule

Perceptron Learning Rule (for Classification)

For a misclassified point $(\mathbf{x}_i, y_i)$:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$$

- $\eta > 0$ is the learning rate.
- $y_i \in \{-1, +1\}$ is the true label.
- Updates only when a mistake is made.
- **Guaranteed to converge** if data is linearly separable.

11.3 Delta Rule (for Regression)

Delta Rule

Minimize the MSE by gradient descent:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y_i - \hat{y}_i) \mathbf{x}_i$$

- Updates proportional to the **error** ($y_i - \hat{y}_i$).
- Works for **continuous outputs** (regression).
- Essentially gradient descent on MSE.

11.4 Activation Functions

Function	Formula	Notes
Step	$f(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$	Used in original perceptron; not differentiable
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$	Output in $(0, 1)$; used for probabilities
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Output in $(-1, 1)$; zero-centered
ReLU	$f(z) = \max(0, z)$	Most popular in deep learning; simple and fast

11.5 Multi-Layer Perceptron (MLP)

MLP Architecture

An MLP consists of:

- **Input layer:** Receives the feature vector $\mathbf{x}$.
- **Hidden layer(s):** One or more layers of neurons that learn intermediate representations.
- **Output layer:** Produces the final prediction.

Each layer computes: $\mathbf{h} = \text{activation}(\mathbf{W}\mathbf{x} + \mathbf{b})$.

With at least **one hidden layer** and non-linear activations, an MLP can approximate any continuous function (**Universal Approximation Theorem**).

11.6 Biological vs. Artificial Neurons

Important Distinction

Aspect	Biological Neuron	Artificial Neuron
Input	Dendrites	Feature vector $\mathbf{x}$
Processing	Cell body (soma)	Weighted sum + activation
Output	Axon	$\hat{y}$
Learning	Synaptic plasticity	Weight updates via gradient descent
Speed	Slow ($\sim$ ms)	Fast (nanoseconds)

Artificial neural networks are **engineering tools**, not biological models.

12 Neural Networks as Function Approximators (L15 – 9 March)

Core Viewpoint: Learning as Function Approximation

In machine learning we seek the **best hypothesis** h from a class $\mathcal{H}$ to approximate the *unknown* true function f :

$$h^* =_{h \in \mathcal{H}} \mathcal{L}(h(\mathbf{x}), \mathbf{y})$$

A **neural network** is one such hypothesis class – its learnable weights $\mathbf{W}$ parametrise $h_{\mathbf{W}}(\mathbf{x})$. Training = finding $\mathbf{W}^*$. Even a perfect network is *not* f itself; it *approximates* f on the available data.

12.1 Why Non-Linearity is Essential

Key Theorem – No Non-Linearity, No Depth

Stacking L linear layers without activations collapses to a **single linear transformation**:

$$\mathbf{W}_L \cdots \mathbf{W}_2 \mathbf{W}_1 \mathbf{x} = \mathbf{W}_{\text{eff}} \mathbf{x}$$

A network of *any depth* is only as expressive as a single layer if no non-linear activation is applied.

Non-linear activations allow the network to learn *composite, curved decision regions*:

- **Layer 1** (input $\rightarrow$ hidden): creates hyperplane partitions of input space.
- **Layer 2** (hidden $\rightarrow$ hidden): combines these into *convex* regions.
- **Deep layers**: intersect and union convex regions to approximate *arbitrary* shapes.

12.2 The XOR Problem Revisited

XOR is the canonical example of a problem **not linearly separable**:

x_1	x_2	XOR
-1	-1	-1
-1	+1	+1
+1	-1	+1
+1	+1	-1

A single perceptron cannot separate the two classes. An MLP solves XOR by computing:

$$\text{XOR}(x_1, x_2) = (x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$$

implementing AND and OR with two hidden neurons and combining them in the output.

12.3 MLP Computation Flow

For a 2-input, 2-hidden-neuron, 1-output MLP (all with sigmoid activations, linear output):

Forward Pass – Explicit Form

Step 1 – Linear transform (hidden layer):

$$\mathbf{p} = \mathbf{W} \mathbf{x}, \quad \mathbf{W} \in \mathbb{R}^{2 \times 2}, \mathbf{x} \in \mathbb{R}^2$$

Step 2 – Non-linear activation:

$$\mathbf{q} = \sigma(\mathbf{p}), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \quad (\text{element-wise})$$

Step 3 – Output linear transform:

$$s = \mathbf{u}^\top \mathbf{q}, \quad \mathbf{u} \in \mathbb{R}^2$$

Step 4 – Output activation (linear for regression, softmax for classification):

$$\hat{y} = \phi(s) = s \quad (\text{regression}), \quad \hat{y}_k = \frac{e^{s_k}}{\sum_j e^{s_j}} \quad (\text{classification})$$

Parameter count for an MLP with bias: 3 inputs, 1 hidden layer with 2 neurons, 1 output neuron with linear activation:

$$\underbrace{(3+1) \times 2}_{=8, \text{ input} \rightarrow \text{hidden}} + \underbrace{(2+1) \times 1}_{=3, \text{ hidden} \rightarrow \text{output}} = 11 \text{ learnable parameters}$$

12.4 Universal Approximation Theorem

Universal Approximation Theorem

An MLP with **at least one hidden layer** and a *non-linear* activation function can approximate **any continuous function** to any desired accuracy, given *sufficient neurons*.

Practical caveat: the theorem guarantees existence, not learnability. Training such a network (optimisation + data) is the real challenge.

12.5 Softmax and Cross-Entropy Loss

Classification Output Pipeline

Softmax converts raw scores $\mathbf{s} = (s_1, \dots, s_K)$ to probabilities:

$$P(y = k | \mathbf{x}) = \frac{e^{s_k}}{\sum_{j=1}^K e^{s_j}}$$

Cross-entropy loss (for true class c):

$$\mathcal{L} = -\log P(y = c | \mathbf{x})$$

Combined **gradient** w.r.t. logit s_k :

$$\frac{\partial \mathcal{L}}{\partial s_k} = P(y = k | \mathbf{x}) - \mathbf{1}[k = c]$$

12.6 Feature Hierarchy in Deep MLPs

A practical interpretation of hidden layers:

- **Early layers:** combine raw sensor measurements (temperature, pixel values) into composite, low-level features.
- **Middle layers:** combine low-level features into mid-level abstractions (edges $\rightarrow$ shapes).

- **Final layers:** reduce to class-specific representations; nearly linearly separable at this stage.

The *weights* determine which combination is useful for the given task – they are learned, not hand-crafted.

Practice & Review – L15

Q1: Conceptual

Question: Why can removing non-linear activations from a deep MLP collapse it to a single layer? Provide a mathematical justification.

Answer: Without activations, each layer is $\mathbf{W}_i \mathbf{x}$; composing L such layers gives $\mathbf{W}_L \cdots \mathbf{W}_1 \mathbf{x} = \mathbf{W}_{\text{eff}} \mathbf{x}$, a single linear transformation. No matter how many layers are added, the function remains linear and cannot learn non-linear decision boundaries.

Q2: Computation

Question: An MLP has 3 inputs, one hidden layer with 4 neurons (sigmoid, no bias), and 1 output neuron (linear, with bias). How many learnable parameters does it have?

Answer: Input→hidden: $3 \times 4 = 12$. Hidden→output: $4 \times 1 = 4$ weights +1 bias = 5. Total: **17**.

Q3: Short Answer

Question: What does the Universal Approximation Theorem guarantee, and what does it *not* guarantee?

Answer: It guarantees that a sufficiently wide single-hidden-layer MLP *can represent* any continuous function. It does **not** guarantee that gradient-based training will *find* those weights, nor that the required width is practical.

Q4: Multiple Choice

Question: Softmax outputs sum to 1. Which modification to the logits *preserves* the softmax output?

A) Adding a constant c to all logits B) Multiplying all logits by $c > 1$ C) Squaring all logits D) None of the above

Answer: A. $e^{s_k+c} / \sum e^{s_j+c} = e^c e^{s_k} / (e^c \sum e^{s_j}) = e^{s_k} / \sum e^{s_j}$. The constant cancels. This is used in the “log-sum-exp trick” for numerical stability.

13 Backpropagation Algorithm (L16 – 12 March)

What is Backpropagation?

Backpropagation is an **efficient algorithm to compute gradients** of the loss $\mathcal{L}$ with respect to *all* learnable parameters of a neural network. It exploits the **chain rule** of calculus and the **reusability** of intermediate partial derivatives computed in the backward direction.

13.1 Neural Network as a Computational Graph

Represent the network as a directed acyclic graph (DAG):

$$\mathbf{x} \xrightarrow{f_1(\mathbf{W}_1)} \mathbf{h}_1 \xrightarrow{f_2} \mathbf{h}_2 \xrightarrow{f_3(\mathbf{W}_3)} \mathbf{h}_3 \xrightarrow{\dots} \hat{y} \xrightarrow{\text{loss}} \mathcal{L}$$

- **Nodes with parameters** (e.g., linear layers $\mathbf{W}$): need $\partial\mathcal{L}/\partial\mathbf{W}$ for gradient descent.
- **Nodes without parameters** (e.g., ReLU, sigmoid): need $\partial\mathcal{L}/\partial\text{input}$ to pass gradients through.

13.2 Forward Pass

For a simple two-layer MLP with inputs x_1, x_2 , hidden neurons, and output:

Forward Pass – Step by Step

$$\mathbf{p} = \mathbf{W}\mathbf{x} \quad (\text{linear}) \quad \mathbf{q} = \sigma(\mathbf{p}) \quad (\text{sigmoid}) \quad s = \mathbf{u}^\top \mathbf{q} \quad (\text{linear}) \quad o = \phi(s) \quad (\text{output})$$

$$\mathcal{L} = (o - y)^2 \quad (\text{MSE loss, single sample})$$

Store all intermediate values $\{\mathbf{p}, \mathbf{q}, s, o\}$ – they are needed in the backward pass.

13.3 Backward Pass and the Chain Rule

Chain Rule for Gradient Computation

To find $\partial\mathcal{L}/\partial u_1$ (weight in the second layer):

$$\frac{\partial\mathcal{L}}{\partial u_1} = \underbrace{\frac{\partial\mathcal{L}}{\partial o}}_{\text{loss} \rightarrow o} \cdot \underbrace{\frac{\partial o}{\partial s}}_{\phi'(s)} \cdot \underbrace{\frac{\partial s}{\partial u_1}}_{q_1}$$

To find $\partial\mathcal{L}/\partial w_{11}$ (weight in the first layer):

$$\frac{\partial\mathcal{L}}{\partial w_{11}} = \frac{\partial\mathcal{L}}{\partial o} \cdot \frac{\partial o}{\partial s} \cdot \frac{\partial s}{\partial q_1} \cdot \frac{\partial q_1}{\partial p_1} \cdot \frac{\partial p_1}{\partial w_{11}} = \frac{\partial\mathcal{L}}{\partial o} \cdot \phi'(s) \cdot u_1 \cdot \sigma'(p_1) \cdot x_1$$

Key insight: the terms $\partial\mathcal{L}/\partial o$ and $\phi'(s)$ computed for u_1 are *reused* for u_2, w_{11}, w_{12} , etc. Computing backward avoids redundant work – that is the essence of backpropagation.

13.4 Derivatives of Common Layers

Layer	Forward	Backward ($\partial\text{output}/\partial\text{input}$)
Linear	$\mathbf{y} = \mathbf{W}\mathbf{x}$	$\partial/\partial\mathbf{x} = \mathbf{W}^\top; \quad \partial/\partial\mathbf{W} = \delta \mathbf{x}^\top$
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$
Tanh	$\tanh(z)$	$1 - \tanh^2(z)$
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$ (sub-gradient at 0)
MSE Loss	$(o - y)^2$	$2(o - y)$
Softmax+XEnt	combined	$P(k) - \mathbf{1}[k = c]$

13.5 Backpropagation Algorithm (General)

Training Loop

1. **Initialise** all weights $\mathbf{W}$ (e.g., randomly).
2. **Sample** a mini-batch $\{(\mathbf{x}_i, y_i)\}_{i=1}^B$ from training data.
3. **Forward pass**: compute all layer outputs and store them.
4. **Compute loss** $\mathcal{L} = \frac{1}{B} \sum_i \ell(\hat{y}_i, y_i)$.
5. **Backward pass**: propagate $\partial\mathcal{L}/\partial\cdot$ backward through the graph using the chain rule, reusing previously computed terms.
6. **Update**: $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}$.
7. Repeat from step 2 until convergence.

Designing custom layers: any block f may be added to a network provided:

- $\partial f(\mathbf{x})/\partial\mathbf{x}$ is computable (differentiable w.r.t. its input).
- If f has parameters $\mathbf{W}$: $\partial f/\partial\mathbf{W}$ is also computable.

Modern frameworks (PyTorch, JAX) automate this via *automatic differentiation*.

13.6 Vanishing and Exploding Gradients

The Vanishing/Exploding Gradient Problem

In a deep network, the gradient of the loss w.r.t. an early-layer weight involves a **long chain of multiplications**:

$$\frac{\partial\mathcal{L}}{\partial w^{(1)}} = \prod_{l=2}^L \frac{\partial \mathbf{h}^{(l)}}{\partial \mathbf{h}^{(l-1)}} \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}}$$

- If each factor < 1 (e.g., sigmoid saturates): product **vanishes** $\rightarrow 0$. Early layers stop learning.
- If each factor > 1 : product **explodes** $\rightarrow \infty$. Training diverges.

13.7 Residual / Skip Connections

Residual Connections (He et al., 2015)

Instead of $\mathbf{h}^{(l+1)} = f(\mathbf{h}^{(l)})$, use:

$$\mathbf{h}^{(l+1)} = f(\mathbf{h}^{(l)}) + \mathbf{h}^{(l)}$$

Gradient effect: differentiating through the addition:

$$\frac{\partial \mathbf{h}^{(l+1)}}{\partial \mathbf{h}^{(l)}} = \frac{\partial f}{\partial \mathbf{h}^{(l)}} + \mathbf{I}$$

The identity term $\mathbf{I}$ provides a **direct path** for gradients to flow backward without decay. This largely solves the vanishing gradient problem and enables training of very deep networks (ResNets, Transformers).

Constraint: $f(\mathbf{h}^{(l)})$ and $\mathbf{h}^{(l)}$ must have the **same dimension** for element-wise addition.

Practice & Review – L16

Q1: Conceptual

Question: Why is the backward pass computed in *reverse* order through the computational graph?

Answer: Because of the chain rule, computing gradients at layer l requires the gradient at layer $l + 1$. By propagating backward (from loss to inputs), each intermediate gradient is computed *once* and reused for all parameters in earlier layers, avoiding redundant computation.

Q2: Derivation

Question: Show that $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Answer:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z)). \quad \square$$

Q3: Multiple Choice

Question: Which technique directly addresses vanishing gradients by ensuring a gradient path exists bypassing deep layers?

A) Dropout B) Batch Normalisation C) Residual Connections D) Weight Decay

Answer: C. Residual connections add the identity mapping, giving $\partial/\partial \mathbf{h} = \partial f/\partial \mathbf{h} + \mathbf{I}$, guaranteeing a gradient magnitude ≥ 1 at minimum.

Q4: Essay

Question: Explain how backpropagation with regularisation works. How does L2 regularisation change the gradient update?

Answer: With L2 regularisation the loss becomes $\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \|\mathbf{W}\|_2^2$. The gradient is $\nabla_{\mathbf{W}} \mathcal{L}_{\text{reg}} = \nabla_{\mathbf{W}} \mathcal{L} + 2\lambda \mathbf{W}$. The update rule becomes $\mathbf{W} \leftarrow \mathbf{W}(1 - 2\eta\lambda) - \eta \nabla_{\mathbf{W}} \mathcal{L}$, which *shrinks* weights at each step (weight decay). The backpropagation algorithm itself is unchanged; only the loss function has an additional term.

14 Neural Network Design & Training (L17 – 23 March)

The Full Training Pipeline

1. Understand the problem: define input $\mathbf{x}$, output $\mathbf{y}$, loss function.
2. Choose architecture (depth, width, layer types).
3. **Initialise** weights.
4. Run forward + backward pass with a chosen optimiser.
5. Monitor training/validation loss; apply regularisation as needed.
6. Tune hyperparameters (learning rate, batch size, architecture).

14.1 Weight Initialisation

Poor initialisation causes slow convergence or vanishing/exploding gradients from the very first step.

Initialisation Strategies

- **Zero initialisation:** all weights = 0. **Fails:** all neurons in a layer compute the same gradient (symmetry problem).
- **Random (Gaussian):** $w \sim \mathcal{N}(0, \sigma^2)$ with small σ . Works for shallow nets.
- **Xavier / Glorot** (for sigmoid/tanh):

$$w \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, +\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

Keeps activation *variance* roughly constant across layers.

- **Kaiming / He** (for ReLU):

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

Accounts for the fact that ReLU zeros out half its inputs on average.

14.2 Gradient Descent Variants

Variant	Batch Size	Properties
Batch GD	Full dataset	Stable gradient; very slow per epoch on large data
Stochastic GD (SGD)	1 sample	Noisy but fast; can escape local minima
Mini-Batch GD	B samples (e.g., 32/128)	Best of both; parallelisable on GPU

Update rule (mini-batch):

$$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{B} \sum_{i=1}^B \nabla_{\mathbf{W}} \mathcal{L}_i$$

14.3 Momentum

SGD with Momentum

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\mathbf{W}} \mathcal{L}$$

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \mathbf{v}_t$$

Momentum $\beta \in [0, 1)$ (typically 0.9) accumulates a velocity vector, **dampening oscillations** and helping escape shallow local minima.

14.4 Regularisation for Neural Networks

Regularisation Techniques

- **L1 / L2 weight decay**: added to the loss; discussed in Lecture 4.
- **Dropout** (Srivastava et al., 2014): during training, randomly set each neuron's output to 0 with probability p (typically 0.5 for hidden layers, 0.1 for input). At test time, scale outputs by $(1 - p)$.
 - Prevents *co-adaptation*: no neuron can rely on another being present.
 - Equivalent to averaging over 2^n sub-networks.
- **Early stopping**: monitor validation loss; stop training when it starts to increase.
- **Data augmentation**: artificially expand training set via transformations (flips, crops, noise). Especially powerful in computer vision.

14.5 Batch Normalisation

Batch Normalisation

For each mini-batch $\mathcal{B}$ and feature dimension j :

$$\hat{x}_j = \frac{x_j - \mu_{\mathcal{B},j}}{\sqrt{\sigma_{\mathcal{B},j}^2 + \epsilon}}, \quad y_j = \gamma_j \hat{x}_j + \beta_j$$

where γ_j, β_j are *learnable* scale and shift parameters.

Benefits:

- Reduces *internal covariate shift* (distribution of activations changes during training).
- Allows higher learning rates; acts as mild regulariser.
- Smooths the loss landscape.

14.6 Learning Rate Scheduling

Scheduler	Rule
Step decay	$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/k \rfloor}$
Exponential decay	$\eta_t = \eta_0 \cdot e^{-\lambda t}$
Cosine annealing	$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$
Warm-up	Start with small η , increase to $\eta_{\max}$, then decay

Practice & Review – L17

Q1: Conceptual

Question: Why does zero initialisation fail for neural networks, but not for linear regression?

Answer: In linear regression there is only one layer; all weights update independently. In an MLP, zero initialisation causes all hidden neurons to be identical (same inputs, same gradients), so they update identically and the network never breaks symmetry – it behaves as if it had a single hidden neuron regardless of width.

Q2: Short Answer

Question: What is dropout, and why is scaling needed at test time?

Answer: Dropout randomly zeros activations with probability p during training, preventing over-reliance on individual neurons. At test time *all* neurons are active, so their expected output is $(1 - p)$ times the training-time expectation. Multiplying by $(1 - p)$ at test time (or equivalently dividing by $(1 - p)$ at train time – *inverted dropout*) corrects this mismatch.

Q3: Multiple Choice

Question: Xavier (Glorot) initialisation is designed for which activation function family?
A) ReLU B) Sigmoid / Tanh C) Leaky ReLU D) Softmax

Answer: **B.** Xavier initialisation targets activations that are symmetric and roughly linear near zero, which matches sigmoid and tanh. ReLU zeroes out negative inputs, so Kaiming initialisation is preferred.

Q4: Essay

Question: Compare Batch Gradient Descent, SGD, and Mini-Batch GD. Which is preferred in practice and why?

Answer: Batch GD computes exact gradients but is prohibitively slow for large datasets. SGD uses a single sample – very noisy, but can escape local minima and is fast per update. Mini-Batch GD averages gradients over B samples (typically 32–512): noise is reduced, GPU parallelism is exploited (matrix operations on a batch are efficient), and it converges more reliably than pure SGD. Mini-batch GD is the default in modern deep learning frameworks.

15 Transfer Learning & Advanced Optimisation (L18 – 26 March)

15.1 Transfer Learning

Core Idea

A model trained on a *source* task (e.g., ImageNet classification) encodes general representations in its early/middle layers. These can be **re-used** for a related *target* task with much less data and compute.

- **Feature extraction:** freeze all pre-trained layers; train only the final classification head on target data.
- **Fine-tuning:** initialise with pre-trained weights; continue training *all* (or later) layers on target data with a small learning rate.
- **When tasks are similar** (e.g., speech recognition → speech emotion recognition): the shared input modality and early representations allow large portions of the network to be reused unchanged.

Practical requirement: normalise / standardise inputs to match the format expected by the pre-trained model (same sampling rate, image resolution, etc.).

Why it works: the first layers of a deep network learn *task-agnostic* features (edges in vision, phonemes in speech); only the final layers are task-specific.

15.2 Advanced Gradient Descent: Motivation

Standard SGD uses only the *first derivative* (gradient). Issues:

- **Flat regions:** gradient ≈ 0 ; learning stalls.
- **Saddle points:** gradient = 0 but not a minimum.
- **Ravines:** loss changes rapidly in one direction but slowly in another; gradient oscillates.

15.3 Second-Order Methods

Newton's Method

Use the second derivative (Hessian $\mathbf{H}$) to rescale the gradient:

$$\mathbf{W} \leftarrow \mathbf{W} - \mathbf{H}^{-1} \nabla_{\mathbf{W}} \mathcal{L}$$

Advantages: accounts for curvature; converges in fewer steps.

Disadvantages: $\mathbf{H}$ has $O(n^2)$ entries (n = number of parameters); inverting it is $O(n^3)$ – computationally prohibitive for modern networks with millions of parameters.

Quasi-Newton methods (e.g., L-BFGS) approximate $\mathbf{H}^{-1}$ without computing it explicitly. Used in some small/medium network applications.

15.4 Adaptive Learning Rate Methods

AdaGrad, RMSProp, Adam

AdaGrad: accumulate squared gradients; scale η per-parameter:

$$G_t = G_{t-1} + g_t^2, \quad \theta \leftarrow \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

Parameters with large historical gradients get *smaller* effective learning rates.

RMSProp: exponential moving average of squared gradients (avoids monotone decay of AdaGrad):

$$G_t = \rho G_{t-1} + (1 - \rho) g_t^2, \quad \theta \leftarrow \theta - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

Adam (Adaptive Moment Estimation):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{1st moment - mean})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{2nd moment - variance})$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{bias correction})$$

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

Defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Adam is the **default optimiser** in most modern deep learning work.

Optimiser	Key Property	When to Use
SGD	Simple, well-understood	Small nets, when fine-grained control needed
SGD+Momentum	Faster convergence, fewer oscillations	General purpose
AdaGrad	Adapts LR per-parameter	Sparse gradients (NLP)
RMSProp	Fixes AdaGrad's monotone decay	RNNs, non-stationary objectives
Adam	Momentum + adaptive LR	Default choice; most architectures

15.5 Practical Training Checklist

Before You Train

1. **Data:** normalise inputs; check class balance; augment if needed.
2. **Architecture:** start simple (fewer layers/params); add complexity if underfitting.
3. **Initialisation:** use Xavier (sigmoid/tanh) or Kaiming (ReLU); or use pre-trained weights.
4. **Optimiser:** start with Adam; switch to SGD+momentum near convergence for potential better generalisation.
5. **Learning rate:** use a scheduler; try warm-up for transformers.
6. **Regularisation:** add dropout / weight decay if validation loss diverges from training loss.
7. **Monitor:** log training/validation loss and metrics every epoch; use early stopping.

Practice & Review – L18

Q1: Conceptual

Question: Explain the difference between feature extraction and fine-tuning in transfer learning. When would you choose each?

Answer: **Feature extraction** freezes all pre-trained weights and only trains a new output head. Use it when the target dataset is *small* and similar to the source – risk of overfitting is high. **Fine-tuning** continues training some or all layers. Use it when the target dataset is larger or the tasks differ more – allows the network to adapt its representations to the target domain.

Q2: Short Answer

Question: Why is the Hessian matrix impractical for training large neural networks?

Answer: For a network with n parameters, the Hessian has n^2 entries. A network with 10^6 parameters would require 10^{12} entries just to store the Hessian, and $O(n^3) = 10^{18}$ operations to invert it – completely infeasible.

Q3: Derivation

Question: Write the Adam update rule and explain the purpose of bias correction.

Answer: See the formulabox above. **Bias correction:** at step $t = 1$, $m_1 = (1 - \beta_1)g_1 \approx 0.1 g_1$ since $\beta_1 = 0.9$ – the estimate is biased toward zero due to initialisation at $m_0 = 0$. Dividing by $(1 - \beta_1^t)$ corrects for this, giving an unbiased estimate of the true first moment especially in early steps.

Q4: Multiple Choice

Question: Which optimiser applies a *different effective learning rate to each parameter* based on the history of its gradients?

A) SGD with momentum B) Vanilla SGD C) Adam D) Newton's method

Answer: **C** (also B to some extent via AdaGrad heritage, but Adam is the canonical answer). Adam maintains per-parameter first and second moment estimates, resulting in an *adaptive* effective learning rate $\eta/\sqrt{\hat{v}_t}$ for each parameter.

Q5: Essay

Question: Describe the full pipeline for applying transfer learning to a speech emotion recognition task, starting from a pre-trained speech recognition model.

Answer: (1) Obtain a pre-trained ASR model (e.g., Wav2Vec 2.0). (2) Normalise target audio to the model's expected sample rate and format. (3) Remove or replace the final classification head (trained for phoneme/word classes) with a new head for emotion classes. (4) Freeze the feature-extraction layers; train only the new head on labelled emotion data for a few epochs. (5) Optionally fine-tune all layers with a small η (e.g., 10^{-5}). (6) Apply dropout and early stopping to prevent overfitting on the typically small emotion dataset.

16 Formula Reference Sheet

All Key Formulas at a Glance

Linear Regression:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b, \quad \mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

MSE Loss:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Ridge Regression:

$$\mathcal{L} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \|\mathbf{w}\|_2^2, \quad \mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

Lasso Regression:

$$\mathcal{L} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \|\mathbf{w}\|_1$$

Gradient Descent Update:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$$

Eigenvalue Equation:

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$$

SVD:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

PCA Projection:

$$\mathbf{x}' = \mathbf{U}_{d'}^\top (\mathbf{x} - \bar{\mathbf{x}})$$

Covariance Matrix:

$$\mathbf{\Sigma} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top$$

Bayes' Theorem:

$$P(\omega | \mathbf{x}) = \frac{P(\mathbf{x} | \omega) P(\omega)}{P(\mathbf{x})}$$

Gaussian PDF (Multivariate):

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2} |\mathbf{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \mathbf{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

MLE for Gaussian:

$$\hat{\mu} = \frac{1}{n} \sum x_i, \quad \hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \hat{\mu})^2$$

Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Softmax:

$$P(y = k | \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_j e^{\mathbf{w}_j^\top \mathbf{x}}}$$

SVM (Soft Margin):

$$\min \frac{1}{2} \|\mathbf{w}\|^2 + C \sum \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$$

RBF Kernel:

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right)$$

Perceptron Update:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i \quad (\text{if misclassified})$$

Delta Rule:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y_i - \hat{y}_i) \mathbf{x}_i$$

Sigmoid Derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Softmax:

$$\hat{p}_k = \frac{e^{s_k}}{\sum_j e^{s_j}}, \quad \frac{\partial \mathcal{L}_{\text{XEnt}}}{\partial s_k} = \hat{p}_k - \mathbf{1}[k = c]$$

Chain Rule (Backprop):

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(l)}} \cdot \frac{\partial \mathbf{h}^{(l)}}{\partial w_{ij}^{(l)}}$$

Residual Connection:

$$\mathbf{h}^{(l+1)} = f(\mathbf{h}^{(l)}) + \mathbf{h}^{(l)}, \quad \frac{\partial \mathbf{h}^{(l+1)}}{\partial \mathbf{h}^{(l)}} = \frac{\partial f}{\partial \mathbf{h}^{(l)}} + \mathbf{I}$$

Xavier Initialisation:

$$w \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, +\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

Kaiming / He Initialisation (ReLU):

$$w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}}}}\right)$$

SGD with Momentum:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\mathbf{w}} \mathcal{L}, \quad \mathbf{W} \leftarrow \mathbf{W} - \eta \mathbf{v}_t$$

Adam Optimiser:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta \leftarrow \theta - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Batch Normalisation:

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

17 Practice Exam Questions

Q1: Conceptual – Supervised vs. Unsupervised Learning

Question: Explain the difference between supervised and unsupervised learning. Give two examples of each.

Answer: Supervised learning uses labelled data to learn a mapping from inputs to outputs (e.g., spam detection, house price prediction). Unsupervised learning finds structure in unlabelled data (e.g., K-means clustering, PCA for dimensionality reduction).

Q2: Computation – Normal Equation

Question: Given 3 data points $(1, 2)$, $(2, 3)$, $(3, 5)$, compute the linear regression weights using the normal equation $\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$.

Hint: Set up $\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}$, $\mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 5 \end{bmatrix}$ and compute step by step.

Q3: L1 vs. L2 Regularization

Question: Compare L1 and L2 regularization. When would you prefer one over the other?

Answer: L1 (Lasso) adds $\lambda \sum |w_j|$ and produces **sparse** models (some weights become exactly zero), making it good for feature selection. L2 (Ridge) adds $\lambda \sum w_j^2$ and shrinks all weights but does not set any to zero, better when all features are relevant. Geometrically, L1 has a diamond constraint region and L2 has a circular one.

Q4: PCA Steps

Question: Describe the steps of PCA. What does each eigenvalue tell you?

Answer: (1) Center the data. (2) Compute covariance matrix. (3) Find eigenvalues/eigenvectors. (4) Sort by decreasing eigenvalue. (5) Project onto top d' eigenvectors. Each eigenvalue represents the **amount of variance** captured by the corresponding principal component.

Q5: SVD Application

Question: How is SVD used in recommendation systems?

Answer: The user-item rating matrix is decomposed as $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. The top- k singular values/vectors capture latent factors (e.g., genre preferences). Missing entries can be predicted from the low-rank approximation, enabling collaborative filtering.

Q6: Bayes' Theorem Application

Question: A disease affects 1% of the population. A test has 95% sensitivity and 5% false positive rate. If a person tests positive, what is the probability they have the disease?

Hint: Use Bayes' theorem with $P(D) = 0.01$, $P(+|D) = 0.95$, $P(+|\neg D) = 0.05$.

$$P(D|+) = \frac{P(+|D) P(D)}{P(+|D) P(D) + P(+|\neg D) P(\neg D)} = \frac{0.95 \times 0.01}{0.95 \times 0.01 + 0.05 \times 0.99} \approx 0.161$$

Q7: SVM Concepts

Question: What are support vectors? What happens when you increase C in a soft-margin SVM?

Answer: Support vectors are the data points closest to (or on the wrong side of) the decision boundary. They “support” (define) the hyperplane. Increasing C penalizes misclassifications more, leading to a **narrower margin** (less tolerance for errors), which can cause overfitting.

Q8: Kernel Trick

Question: Explain the kernel trick. Why is it useful?

Answer: The kernel trick computes dot products in a high-dimensional feature space **without explicitly** computing the mapping $\phi(\mathbf{x})$. This allows SVMs to learn non-linear decision boundaries efficiently. For example, the RBF kernel $K(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2 / 2\sigma^2)$ maps to an infinite-dimensional space.

Q9: MLE

Question: Derive the maximum likelihood estimate for the mean of a Gaussian distribution.

Answer: The log-likelihood is $\ell(\mu) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum (x_i - \mu)^2$. Differentiating w.r.t. μ and setting to zero: $\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum (x_i - \mu) = 0 \Rightarrow \hat{\mu} = \frac{1}{n} \sum x_i$, which is the sample mean.

Q10: EM Algorithm

Question: Describe the EM algorithm for GMMs. Why might it converge to different solutions?

Answer: E-Step: compute responsibilities γ_{ik} (soft assignments of points to clusters). M-Step: update means, covariances, and mixing coefficients using weighted data. Repeat until convergence. EM converges to a **local maximum**, and different random initializations lead to different local maxima, hence different solutions.

Q11: Neural Networks

Question: Why can't a single-layer perceptron learn XOR? How do we fix this?

Answer: XOR is **not linearly separable** — no single hyperplane can separate the two classes. A single-layer perceptron can only learn linear decision boundaries. **Fix:** Use a **multi-layer perceptron** (MLP) with at least one hidden layer and non-linear activations, or use a kernel method that maps the data to a higher-dimensional space where it becomes linearly separable.

Q12: Bias–Variance Trade-off

Question: Explain the bias–variance trade-off. How does regularization help?

Answer: **Bias** = error from overly simplistic model assumptions (underfitting). **Variance** = error from sensitivity to training data fluctuations (overfitting). Total error $\approx \text{Bias}^2 + \text{Variance} + \text{Irreducible noise}$. Regularization **increases bias slightly** but **reduces variance significantly**, often improving overall generalization.

Q13: Backpropagation

Question: Describe the forward pass and backward pass in a 3-layer MLP. What is stored during the forward pass and why?

Answer: Forward pass: input propagates layer by layer, computing $\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$ at each layer; the final output is compared with the target to compute $\mathcal{L}$. **Stored:** all pre-activation values $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)}$ and activations $\mathbf{h}^{(l)}$ – these are needed to compute derivatives $\sigma'(\mathbf{z}^{(l)})$ during the backward pass. **Backward pass:** compute $\partial\mathcal{L}/\partial\mathbf{W}^{(l)}$ by propagating the gradient from the output layer backward, multiplying by the chain-rule terms at each step and reusing already-computed partial derivatives.

Q14: Vanishing Gradients

Question: A 20-layer sigmoid network fails to improve validation accuracy after the first 5 layers are trained. What is the likely cause, and what are two architectural solutions?

Answer: Cause: vanishing gradients. Sigmoid saturates; $\sigma' \leq 0.25$, so the gradient shrinks by at least $4\times$ per layer. After 20 layers the gradient at layer 1 is $\leq (0.25)^{20} \approx 10^{-12}$ – effectively zero. **Solutions:** (1) Replace sigmoid with **ReLU** activations ($\sigma'(z) = 1$ for $z > 0$, no saturation for positive inputs). (2) Add **residual/skip connections**: $\mathbf{h}^{(l+1)} = f(\mathbf{h}^{(l)}) + \mathbf{h}^{(l)}$, which injects an identity gradient path bypassing each block.

Q15: Transfer Learning

Question: You have 500 labelled chest X-ray images and want to train a classifier. A large model pre-trained on 1M natural images is available. Describe your strategy.

Answer: Use **transfer learning with feature extraction / fine-tuning**: (1) Download the pre-trained model (e.g., ResNet-50 trained on ImageNet). (2) Replace the final fully connected layer with a new head for binary classification (normal/abnormal). (3) With only 500 samples, **freeze all pre-trained layers** and train only the new head to avoid overfitting. (4) Apply **data augmentation** (random flips, rotations, brightness) to expand the effective training set. (5) Use **dropout** in the new head and **early stopping** on validation loss. (6) Optionally, after the head converges, **fine-tune** the last 1–2 blocks with a very small $\eta \approx 10^{-5}$.

Q16: Adam vs. SGD

Question: When would you prefer SGD+Momentum over Adam? What is the key trade-off?

Answer: Adam converges faster because it adapts the learning rate per-parameter. However, studies show that SGD+Momentum can generalise *better* (reach flatter minima) when given enough epochs, particularly in computer vision. Adam may converge to sharp minima that generalise poorly. In practice: use Adam for quick prototyping and transformers/NLP; switch to SGD+Momentum (with a cosine LR schedule) for final training of CNNs when maximum test accuracy is required.

18 Quick-Reference: Topic-by-Lecture Map

Date	Lecture #	Topics Covered
5 Jan	L01	Course overview, supervised/unsupervised learning, nearest neighbour, linear classifier
8 Jan	L02	Data representation, handcrafted vs. learned features, classification vs. regression
12 Jan	L03	Linear regression, loss functions (MSE), gradient descent, closed-form solution, bias–variance
19 Jan	L04	Optimization, regularization (L1 & L2), Lasso vs. Ridge, geometric interpretation
22 Jan	L05	Matrix rank, eigenvalues/eigenvectors, SVD, low-rank approximation
27 Jan	L06	PCA (intro), covariance matrix, eigenvector projection, data compression
29 Jan	L07	Course review: supervised learning, train/val/test, overfitting, performance metrics
2 Feb	L08	PCA (deep dive), SVD applications, LSI, word embeddings, matrix factorization
5 Feb	L09	Bayesian decision theory, normal distributions, Bayes’ theorem, decision boundaries
9 Feb	L10	MLE, Gaussian Mixture Models, EM algorithm, K-means clustering
12 Feb	L11	K-means, Naive Bayes, logistic regression, softmax, loss functions
16 Feb	L12	SVM (hard & soft margin), kernel trick, polynomial/RBF kernels, convex optimization
19 Feb	L13	Practical ML: cross-validation, regularization, overfitting, hyperparameter tuning
23 Feb	L14	Neural networks: perceptron, delta rule, activation functions, MLP, biological vs. artificial
9 Mar	L15	NNs as function approximators, XOR, MLP computation flow, softmax/cross-entropy, UAT
12 Mar	L16	Backpropagation: computational graphs, forward/backward pass, chain rule, vanishing gradients, residual connections
23 Mar	L17	NN design & training: weight initialisation (Xavier, Kaiming), SGD variants, dropout, batch norm, LR scheduling
26 Mar	L18	Transfer learning, fine-tuning, second-order methods (Newton/Hessian), Adam, adaptive LR optimisers

19 Glossary of Key Terms

Term	Definition
Bias	Systematic error from incorrect model assumptions (underfitting).
Classification	Predicting a discrete class label.
Clustering	Grouping similar data points (unsupervised).
Covariance Matrix	Matrix summarizing variance/covariance between all pairs of features.
Cross-Entropy	Loss function for classification tasks.
Cross-Validation	Technique for evaluating models using multiple train/test splits.
Dimensionality Reduction	Reducing the number of features/dimensions.
Eigenvalue	Scalar λ such that $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$.
Eigenvector	Vector $\mathbf{v}$ unchanged in direction by transformation $\mathbf{A}$.
EM Algorithm	Iterative method for MLE with latent variables (E-step + M-step).
Feature	A measurable property of a data point.
Gradient Descent	Iterative optimization: move in direction of steepest descent.
GMM	Gaussian Mixture Model: data modeled as mixture of Gaussians.
Hyperparameter	Parameter set before training (not learned from data).
Kernel	Function computing inner products in a transformed feature space.
Lasso (L1)	Regularization that promotes sparsity.
Likelihood	Probability of data given model parameters.
MLE	Maximum Likelihood Estimation.
MSE	Mean Squared Error.
Naive Bayes	Classifier assuming feature independence given class.
Normal Equation	Closed-form solution for linear regression.
Overfitting	Model memorizes training data; poor generalization.
PCA	Principal Component Analysis: unsupervised dimensionality reduction.
Perceptron	Simplest neural network model.
Posterior	Updated probability after observing evidence.
Prior	Probability before observing evidence.
Regression	Predicting a continuous value.
Regularization	Penalty on model complexity to reduce overfitting.
ReLU	Rectified Linear Unit: $\max(0, z)$.
Ridge (L2)	Regularization that shrinks all weights.
Sigmoid	Logistic function mapping $\mathbb{R} \rightarrow (0, 1)$.
Singular Value	Non-negative value in $\mathbf{\Sigma}$ of SVD.
Softmax	Generalization of sigmoid for multi-class probabilities.
Support Vector	Data point on or inside the SVM margin boundary.
SVD	Singular Value Decomposition: $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$.
SVM	Support Vector Machine: max-margin classifier.
Underfitting	Model too simple to capture data patterns.
Variance	Error from sensitivity to training data fluctuations (overfitting).

Neural Network / Deep Learning Terms (L15–L18)

Adam	Adaptive moment estimation optimiser; maintains per-parameter momentum and variance estimates.
Backpropagation	Algorithm to compute gradients via chain rule in a computational graph, traversed backward.
Batch Normalisation	Normalises layer activations per mini-batch; learnable scale γ and shift β . ⁴⁷
Computational Graph	DAG representation of a neural network where nodes are operations and edges are data flow.
Dropout	Regularization: randomly zero activations during training; pre-